THE AERIAL GUARDIAN

Multi-Object Person Detection & Tracking Pipeline for UAVs

Technical Summary Report

Hardware: NVIDIA GeForce GTX 1050 Ti (4GB VRAM) | Dataset: VisDrone MOT Validation Set

1. Architecture Overview

The pipeline is designed around a core principle: maximum detection recall for small aerial objects while remaining lightweight enough for drone-class hardware. The full pipeline is:

Stage	Component	Purpose
Preprocessing	CLAHE (Contrast Limited Adaptive Histogram Equalization)	Corrects haze and low contrast in drone footage
Tiling	Custom overlapping 640x640 tile slicer	Zooms into tiny persons invisible at full resolution
Detection	YOLOv8s (Small variant, ~22MB)	Detects persons in each tile
Deduplication	Cross-tile NMS (IoU=0.4)	Removes duplicate boxes from overlapping tiles
Post-filtering	Aspect ratio filter (h/w > 0.75)	Removes bicycles and vehicles misdetected as persons
Tracking	Custom Centroid Tracker (KF)	Assigns persistent IDs across frames
Visualization	Trajectory tail renderer	Draws fading colored trails per tracked person

2. Small Object Detection Strategy

2.1 The Core Problem

VisDrone footage is captured at altitudes of 30–100 m, resulting in people being reduced to 5–20-pixel regions in a 1920 × 1080 frame. Standard YOLO inference at any input size struggles because the model's receptive field is tuned for objects that occupy a meaningful portion of the image. A seated or crouching person at altitude may occupy less than 0.01% of the frame area.

2.2 CLAHE Preprocessing

Drone footage frequently suffers from atmospheric haze, overexposure, and flat contrast, all of which reduce the visibility of small objects. CLAHE is applied in the LAB color space, enhancing only the L (luminance) channel. This boosts local contrast without distorting colors, making tiny individuals more distinguishable from background textures.

Key parameters chosen:

- clipLimit = 2.0 - prevents over-amplification of noise in flat sky regions
- tileGridSize = (8, 8) - balances local vs global enhancement

2.3 Overlapping Tile-Based Inference (Custom SAHI-style)

The primary innovation for small object detection is a custom overlapping tile slicer. Each full frame is divided into 640x640 tiles with 100px overlap. YOLO runs on each tile independently at native resolution, effectively zooming in 3x on the original frame content.

Standard YOLOv8n misses small people at drone altitude because they occupy only a few pixels. I integrated SAHI (Slicing Aided Hyper Inference), which divides each frame into overlapping 512×512 tiles, runs inference on each tile, and then merges detections. This directly addresses the small-scale object problem inherent to aerial footage, without increasing model size.

Parameter	Value	Rationale
Tile size	640 x 640 px	Matches YOLOv8 native training resolution
Overlap	100 px	Prevents persons at tile boundaries from being missed
Tiles per frame (1920x1080)	~8 tiles	Full coverage with minimal redundancy
Per-tile confidence	0.20	Balanced, catches seated persons without excessive false positives
Per-tile IOU threshold	0.45	Stricter NMS within each tile

After all tiles are processed, detections are mapped back to full-frame coordinates, and a global NMS pass (IoU=0.4) removes duplicate boxes that appear in the 100px overlap zone between adjacent tiles.

2.4 Aspect Ratio Filtering

A critical observation from drone footage: individuals occupy tall or squarish bounding boxes (height/width > **0.75**), while bicycles, motorcycles, and cars produce wide boxes (height/width < 0.75). A simple aspect ratio filter removes these false positives with zero additional compute cost:

Object	Typical h/w ratio	Filter decision
Standing person	~2.5	Keep
Sitting person	~1.0	Keep
Crouching person	~0.9	Keep
Bicycle	~0.7	Reject

3. Multi-Object Tracking & ID Consistency

3.1 Why Custom Centroid Tracker over DeepSORT

The challenge explicitly requires a lightweight solution under 300MB. DeepSORT adds a separate Re-Identification (ReID) neural network (~100 MB) to extract appearance features for each detection. The implemented solution uses a custom centroid-based tracker with a track buffer, no additional model, no additional VRAM, and no extra dependencies beyond NumPy.

Tracker	Extra Model	VRAM Impact	Speed	Suitable
Custom Centroid Tracker	None	0 MB additional	~31 FPS	Yes
DeepSORT	ReID CNN (~100MB)	+1-2 GB VRAM	~15 FPS	No, too heavy

3.2 How ID Switching is Addressed

Drone ego motion is the primary cause of ID switching; the camera moves, making every object appear to shift simultaneously, even when they are stationary. The following measures are implemented:

- Track buffer (15 frames): Lost tracks are kept alive for 15 frames before being dropped. This handles brief occlusions behind trees, poles, or other persons and prevents unnecessary ID reassignments when a person temporarily exits the detection window due to camera movement.
- Centroid matching with max_dist threshold: New detections are matched to existing tracks by the nearest centroid distance (max 60px). This method is robust to the small inter-frame displacements caused by slow drone drift.
- Custom centroid tracker (no external library): Detections are matched to existing tracks by nearest centroid distance (max_dist=60px). This is implemented entirely in NumPy, with no dependencies on ByteTrack or a Kalman filter, keeping the solution fully self-contained and lightweight.
- Frame skipping with cached results: Detection runs every 2nd frame. When skipped frames occur, the last-known bounding boxes are reused, and the tracker continues using predicted positions, maintaining visual continuity at double the FPS.

4. Optimization & Performance

4.1 Measured Performance

Metric	Value
Average FPS (full pipeline)	~25-35 FPS
Hardware	NVIDIA GeForce GTX 1050 Ti (4GB VRAM)
Model size (YOLOv8s)	~22 MB
Input resolution per tile	640 x 640
Precision mode	FP16 (half precision) *model.overrides["half"] = True # FP16 inference*

4.2 Optimization Techniques Applied

- FP16 Half Precision: All YOLO inference runs in half precision (16-bit floats). This reduces VRAM usage from ~2GB to ~1.5GB and provides a 20-30% speed improvement on CUDA hardware with negligible accuracy loss. I balanced inference speed and accuracy through four lightweight optimizations: CLAHE preprocessing for drone haze, FP16 half-precision inference, adaptive image sizing based on scene density, and frame-skip with result caching. This pushed FPS without increasing model size but decreased the accuracy.
- Frame skipping (DETECT_EVERY=4): tiled inference is computationally expensive (~8 YOLO forward passes per frame). Running detection on every 2nd frame and caching the results approximately doubles the effective FPS. The tracker interpolates positions on skipped frames.
- Adaptive image sizing: When fewer people are detected in the previous frame, the pipeline can reduce **imgsz** to speed up processing in sparse scenes.
- YOLOv8s (Small): The small variant (22MB) was chosen over nano for better accuracy while remaining faster than medium/large variants. This is the optimal trade-off for the 1050 Ti's 4GB VRAM constraint.

5. Edge Hardware Adaptation (NVIDIA Jetson)

5.1 Target Platform

The NVIDIA Jetson Nano and Jetson Orin Nano are the most common drone-mounted edge inference platforms, with 4-8GB shared CPU/GPU memory and TensorRT support.

5.2 Adaptation Steps

Step 1 — Export to TensorRT

Convert the YOLOv8s model to TensorRT INT8 format. This quantizes weights from 32-bit to 8-bit integers, reducing the model size to ~6MB and achieving a 3-5x speedup on Jetson hardware:

```
from ultralytics import YOLO
model = YOLO('yolov8s.pt')
model.export(format='engine', int8=True, imgsz=640)
```

Step 2 — Reduce Tile Count

On Jetson Nano, running 8 tiles per frame is too slow. Reduce the number of tiles to 4 by either increasing the tile size or reducing the overlap.

```
# Jetson-optimized: 4 tiles instead of 8
tile_size=960, overlap=50
```

Step 3 — Reduce Frame Rate

Drone MOT does not require 30 FPS tracking. At 10-15 FPS, ByteTrack still maintains consistent IDs. Run DETECT_EVERY=3 to process every third frame.

Platform	Method	Expected FPS	Model Size
GTX 1050 Ti (dev)	FP16 PyTorch + tiling	25-35 FPS	22 MB
Jetson Orin Nano	TensorRT INT8 + 4 tiles	15-20 FPS	~6 MB
Jetson Nano	TensorRT INT8 + 4 tiles + skip=3	8-12 FPS	~6 MB

6. Engineering Trade-offs Summary

Decision	Trade-off Made	Reasoning
YOLOv8s over YOLOv8n	Slightly slower, more accurate	Small objects need a stronger feature extractor
Tiling over large imgsz	Slower per-frame, better small detection	The core problem is tiny persons, not speed
conf=0.20 over conf=0.10	Misses some tiny persons, fewer false positives	Eliminates bicycle/vehicle misdetections
FP16 inference	Negligible accuracy loss, 25% faster	Free speed gain on any CUDA GPU
Frame skip every 2	Slight position lag, 2x FPS	Acceptable for MOT at drone altitudes

7. What Was Added Beyond Off-the-Shelf

The challenge specifically asks for additions beyond simply running existing models. The following were implemented from scratch on top of YOLOv8 + ByteTrack:

- **Custom overlapping tile slicer**: slices frames into 640x640 tiles with configurable overlap, maps detections back to full-frame coordinates, and runs global NMS across tiles. This directly addresses the core drone small-object problem.
- **Aspect ratio post-filter:** A geometric heuristic that filters false positive detections based on bounding box shape. Effectively removes bicycles and vehicles without any additional models or computing.
- **CLAHE preprocessing in LAB space:** Applied specifically to the luminance channel only, preserving color fidelity while boosting contrast for hazy drone footage.
- **Track buffer with centroid matching:** A custom lightweight tracker layer on top of ByteTrack that keeps lost IDs alive through brief occlusions, specifically tuned for the slow inter-frame motion typical of drone platforms.
- **Fading trajectory tail renderer:** Visualizes the last 40 positions of each tracked person with a colored, thickness-graduated line—thicker at the current position, fading toward the oldest point.

8. References:

1. https://docs.ultralytics.com/guides/sahi-tiled-inference/
2. https://docs.ultralytics.com/datasets/detect/visdrone/
3. https://docs.ultralytics.com/modes/predict/#key-features-of-predict-mode

Pipeline tested on VisDrone2019-MOT-val | YOLOv8s + Custom Tiling | GTX 1050 Ti